#	Milestone	Generative AI Guidelines	Due	Mark Release
4	Solution Design & Implementation Part I: Relational	D: AI Task Completion, Human Evaluation[1]	Nov. 4, Wed., AoE[2]	Nov. 9, Mon.

This course is *project driven*. Students will work in teams to define, develop, and complete a project involving data modeling and representation over the course of one semester. The course project has two main objectives:

- Hands-on experience: Gain practical experience in analyzing and modeling real-world data, while developing skills in data representation, schema design, querying, implementation, evaluation, and teamwork.
- Research experience: Develop an understanding of the research *process*, from identifying a problem and reviewing existing approaches to designing, implementing, and evaluating different representations of data in the same domain.

Projects should aim to advance the state of the art, rather than simply reproduce an existing solution. This provides students with an opportunity to conduct meaningful project while completing the course requirements.

A data modeling project typically starts with a real-world domain and an information need. In this sense, a data modeling project may resemble a software or database project; however, there are important differences. A software project may primarily focus on implementing a functional system, while a data modeling project in this course focuses on understanding the structure and meaning of information and determining how that information can be represented, stored, and queried effectively for globally shared, accessible, and interoperable representations that can support Knowledge Graphs, the Giant Global Graph (GGG), and Linked Open Data[3].

In this course, students will progressively explore different approaches to representing the same real-world domain. The project will begin with conceptual data modeling, including the identification of entities, attributes, and relationships and their representation using an Entity-Relationship Diagram (ERD). The conceptual model will then be transformed into a relational model (logical level of data modeling), including relational tables, keys, and constraints, and implemented and queried using SQL. The project will subsequently extend the representation toward semantic and knowledge representation, using RDF, RDFS, and OWL, and will explore the storage and querying of the resulting knowledge graph using a triple store and SPARQL:

Real-world domain → ERD → Relational Schema → SQL → RDF/OWL → Triple Store → SPARQL

The goal is not simply to implement these technologies independently. Rather, students should understand how different data models represent information and relationships, how one representation can be transformed into another, what information and semantics may be preserved or lost during such transformations, and how different representations affect the types of questions that can be answered.

Roughly, the data modeling project in this course consists of the following milestones (phases):

1. Project Definition & Team Formation (Proposal)

[1] AI is used to complete certain elements of the task, with students providing discussion or commentary on the AI-generated content. This level requires critical engagement with AI-generated content and evaluating its output (e.g. critique or evaluation of AI-generated analysis). You will use AI to complete specific tasks in your assessment. Any AI-created content must be cited.
[2] https://www.timeanddate.com/time/zones/aoe
[3] https://www.dbpedia.org/

2. Background & Related Work (Literature Review)
3. Project Presentation: 3-Minute Talk (3MT)
4. Solution Design & Implementation (Data Modeling & Representation): Part I: Relational
5. Solution Design & Implementation (Data Modeling & Representation): Part II: Semantic Web
6. Project Demostration: 10-Minute Talk (10MT)

A project manual has been prepared to guide students through each milestone. Each milestone breaks the project into a series of smaller steps and deliverables. The current manual covers **Part I: Relational** of the Milestone 4: Solution Design & Implementation, also referred to as Data Modeling & Representation.

4.1. Logical and Physical[4] Modeling using Relational Databased Management Systems (RDBMS)

To store real-world instances of entities and their relationships, we first convert the Entity-Relationship (ER) diagram into a relational schema consisting of relations (tables)[5]. We then store information about entities and their relationships as tuples (rows or records) that conform to the corresponding relation schemas. The details of ER-to-relational mapping have been discussed in class lectures. In summary:

- Each entity set is mapped to a new relation.
- Multivalued attributes are mapped to separate relations, typically linked to the original relation (table) through a Foreign Key (FK).
- Many-to-one relationships are represented using Foreign Key attributes in the relation on the many side.
- Many-to-many relationships are represented using a separate relation containing Foreign Keys referencing the participating entity sets.
- One-to-one relationships can be represented using a Foreign Key in one of the participating relations or a separate relation containing Foreign Keys to both participating entity sets.
- Primary key attributes of entity sets, including natural or business keys, are preserved as Candidate Keys (CKs) and must satisfy the appropriate uniqueness and mandatory-value constraints.
- One-to-one relationships may introduce additional Candidate Keys on the relevant Foreign Key attributes.
- Each relation must have a Primary Key. For this course, use an autogenerated surrogate key named `Id`.
- For IS-A (specialization/generalization) relationships, a common approach is to create separate relations for the parent and child entity sets and link them using Foreign Keys. Alternative mapping strategies may be appropriate depending on the design requirements.

4.2. Choosing a Relational Database Management System (RDBMS)

After converting the ER diagram into a relational schema, we need a database management system (DBMS) to persist, manage, and retrieve the data. A relational database management system (RDBMS) supports relations (tables) and operations on the stored data and can be used in two different deployment configurations:

- In a server-based configuration, a database server manages the data, and client applications connect to it to create and manipulate databases, tables, and records.

[4] **Terminology Note:** Strictly speaking, the physical level modeling is about how data is stored and organized on physical storage media, including storage-dependant file structures and access methods. SQL statements do not directly specify these physical storage details. In this course, however, we use the term *physical level* to refer to translating the logical data model into a working database using SQL statements. This practical focus allows us to explore database implementation without delving into the underlying storage mechanisms managed by the DBMS.

[5] For practical purposes in this manual, the terms 'entity set', 'relation', and 'table' are used interchangeably; 'entity', 'tuple', 'row', and 'record' refer to the same concept; and 'domain', 'column', and 'attribute' are treated as equivalent terms. Strictly speaking, these terms originate from different levels of data modeling, but we use them interchangeably here for simplicity of implementation.

- In a local or embedded configuration, the database engine operates on the same computer as the application that uses it.

For this course, a lightweight local RDBMS, such as SQLite, is sufficient for evaluation purposes. It allows to create tables, insert records, and retrieve data without setting up a separate database server. Although this manual uses SQLite, you may use any lightweight local RDBMS. However, your repository must document the installation and setup instructions required for an end user to run and work with your database.

SQLite command-line tools can be downloaded for an operating system from the official website[6]:

1. Extract the downloaded files if necessary.
2. Open a terminal (macOS/Linux) or Command Prompt/PowerShell (Windows).
3. Verify that the `sqlite3` command is available. If it is not, navigate to the extracted directory or configure your system's executable search path.

Structured Query Language (SQL). Relational database management systems use SQL to create a database, its relations, specify constraints like Primary Key, Foreign Key, and Candiate Key, and insert, modify and retrieve tuples (records). SQL is standardized by ISO[7] and IEC[8]. The standard provides a common foundation for relational database operations to make SQL statements interchangeable across all RDBMSs (although individual database systems may differ in supported features, syntax, and implementation details.) The latest SQL standard is SQL:2023 or ISO/IEC 9075:2023 in three parts[9]. However, for common, everyday development, also for this course, the SQL:1999 (specifically SQL-92 and SQL:1999 core features) are sufficient.

SQL statements can be broadly classified into two categories: Data Definition Language (DDL) and Data Manipulation Language (DML). DDL is used to define and modify the database schema, including creating relations (tables) and specifying domains (columns), data types, and constraints. Common DDL commands include `CREATE`, `ALTER`, and `DROP`. DML is used to manipulate the data stored in these tables, including inserting, updating, and deleting records using commands such as `INSERT`, `UPDATE`, `DELETE`, `SELECT`. The `SELECT` statement is used to retrieve stored data and is often taught alongside DML (although some classifications treat it separately).

4.3. Creating a Database for the Team Project (DDL)

SQL standard requires an explicit SQL statement to create a database. For example, in PostgreSQL, MySQL, and Microsoft SQL Server, the command is generally:

```
CREATE DATABASE Movision;
```

After creating the database, it should be selected to work with. The exact command used to connect to a database depends on the RDBMS and the client application. For example, in MySQL or Microsoft SQL Server, the newly created database is selected using:

```
USE Movision;
```

[6] https://www.sqlite.org/download.html
[7] https://www.iso.org/about
[8] https://www.iec.ch/who-we-are
[9] https://webstore.ansi.org/preview-pages/ISO/preview_ISO+IEC+9075-1-2023.pdf

These examples illustrate that, although `CREATE DATABASE` is widely supported, database connection and management commands can vary across systems. SQLite does not require this step because the database is represented by a local file. When you open or create a database file, SQLite makes that file available for subsequent database operations as follows:

```
> sqlite3 Movision.db
```

First, replace `Movision.db` with a meaningful database filename that reflects your team's project. Then, for each entity set in your ER diagram, create a corresponding relation (table) in the database. Depending on your ER-to-Relational mapping, you may also need additional relations to represent multivalued attributes, many-many, and one-one relationships. Each relation (table) should have the following:

1. Domain (Attributes, Columns): Create a column for each attribute, with an appropriate data type and a decision about whether its value can be `NULL`.
2. Surrogate Primary Key: Add an autogenerated integer column named `Id` to uniquely identify each tuple (row or record).
3. Candidate Keys: Identify attributes or combinations of attributes that must uniquely identify tuples, especially for Primary Keys at ER diagram for natural (business) keys.
4. Foreign Keys: Identify relationships with other tables and define Foreign Keys as required by the ER-to-Relation mapping.

Suppose we want to create a table for the `Director` relation. We will follow these steps:

1. Create a table called `Director`.
2. Add columns called `Id` and `Name`.
3. Define `Id` as an autogenerated integer primary key.
4. Define `Name` as a mandatory text attribute.
5. Add a `PRIMARY KEY` constraint on `Id`.
6. If in ER diagram director's name is Primary Key, add `UNIQUE` constraint on `Name` as a Candiate Key.

The resulting SQL statement is:

```
CREATE TABLE Director (
    Id INTEGER PRIMARY KEY AUTOINCREMENT,
    Name VARCHAR(255) NOT NULL,
    CONSTRAINT UK_Name UNIQUE (Name)
);
```

Consider the `Movie` entity set, which participates in a one-many relationship with `Director`: one director may direct multiple movies, while each movie is associated with one director. To represent this relationship, we place a Foreign Key in the `Movie` table, referencing the `Director` table. The Foreign Key is placed on the *many* side of the relationship.

```
CREATE TABLE Movie (
    Id INTEGER PRIMARY KEY AUTOINCREMENT,
    Title VARCHAR(255) NOT NULL,
    Genre VARCHAR(255) NOT NULL,
    DirectorId INTEGER NOT NULL,
```

```
    CONSTRAINT FK_Movie_DirectorId_2_Director_Id
        FOREIGN KEY (DirectorId)
        REFERENCES Director(Id)
);
```

The `DirectorId` stores the identifier of a director associated with each movie. For example, if a movie has `DirectorId=3`, we can retrieve the corresponding director by looking up the record whose `Id` is `3` in the `Director` table.

If a movie may have no assigned director, `DirectorId` could instead be nullable. The appropriate choice depends on the assumptions and constraints of your team project. For a many-many relationship, such as movies having multiple directors, a separate relation (table) containing Foreign Keys to both `Movie` and `Director` would be needed:

```
CREATE TABLE MovieDirector (
    Id INTEGER PRIMARY KEY AUTOINCREMENT,
    MovieId INTEGER NOT NULL,
    DirectorId INTEGER NOT NULL,

    CONSTRAINT FK_MovieDirector_MovieId_2_Movie_Id
        FOREIGN KEY (MovieId)
        REFERENCES Movie(Id)

    CONSTRAINT FK_MovieDirector_DirectorId_2_Director_Id
        FOREIGN KEY (DirectorId)
        REFERENCES Director(Id)
);
```

Autogenerating Primary Keys. Most RDBMSs provide mechanisms for generating identifiers automatically. In SQLite, a column declared as `INTEGER PRIMARY KEY` is an alias for the row identifier (`ROWID`). When an insert does not provide an `Id`, SQLite automatically assigns an unused integer, usually one greater than the largest existing row identifier. The optional `AUTOINCREMENT` keyword changes this behavior by preventing SQLite from reusing previously allocated row identifiers and generating increasing identifiers explicit.

Naming Conventions. Throughout this course, we use PascalCase as naming convention for relation (table) and domain (column) names, such as `Director`, `Movie`, and `DirectorId`. However, SQL keywords are written in uppercase, such as `CREATE TABLE`, `PRIMARY KEY`, and `FOREIGN KEY`. These are conventions for readability; SQL keywords are generally case-insensitive. We also use the following naming conventions for constraints:

- `PK_` for Primary Key, where explicitly named.
- `UK_` for Candidate Key (unique).
- `FK_` for Foreign Key.

For example, `UK_Name` identifies a unique constraint on `Name`. For Foreign Key naming convention, we use:

```
FK_SourceTableName_ForeignKeyColumn_2_TargetTableName_TargetKeyColumn
```

For the above example, the source table is `Movie`, the Foreign Key is `DirectorId`, and the target table and key are `Director` and `Id`, respectively. Therefore:

```
FK_Movie_DirectorId_2_Director_Id
```

Enabling Foreign-Key Enforcement in SQLite. SQLite supports Foreign Key, but enforcement must be enabled for each database. After opening your database, execute:

```
PRAGMA foreign_keys;
```

If the result is `0`, Foreign Key enforcement is disabled. Enable it by executing:

```
PRAGMA foreign_keys = ON;
```

Then verify the setting:

```
PRAGMA foreign_keys;
```

The expected result is `1`, indicating that Foreign Key enforcement is enabled. If the result remains `0`, verify that you are using a compatible SQLite build. If Foreign Key enforcement is unavailable in your SQLite build, investigate the SQLite version and compilation options.

Completing the Relational Schema. Apply the same principles to every entity in your ER-to-Relational mapping. Your final SQL script should contain all the `CREATE TABLE` statements needed to reproduce the relational schema. It should execute successfully on a fresh database and implement the relations, attributes, and keys in your ER-to-Relational mapping.

4.4. Populating the Database (DML)

Once relations (tables) are created, we can populate the information about entities and their relationships into them. Suppose we want to add the following movies to the `Movision` database:

Director	Title	Genre
Hitchcock	*Psycho*	*Horror*
Hitchcock	*The Birds*	*Horror*
Scorsese	*Raging Bull*	*Drama*
Kurosawa	*Seven Samurai*	*Samurai*
Wilder	*Sunset Boulevard*	*Noir*
Spielberg	*Schindler's List*	*Drama*
Welles	*Citizen Kane*	*Noir*

Each movie is going to require two entries: one in the `Movie` table, and one in the `Director` table. However, we do not want to input each director's information more than once, and we need to know the director's `Id` before we can insert the movie. This means that there are three stages to inserting a movie:

1. Check the `Director` table and see if the director for the movie already exists or not;
2. If the director is already stored in the `Director` table, just retrieve his/her `Id`
3. If the director is not stored in the `Director` table, insert it and retrieve its `Id`
4. Insert the movie into `Movie` table and for `DirectorId`, use the `Id` that was retrieved in step 2 or 3.

INSERT. To add a new director to `Director` table, SQL statement is:

```
INSERT INTO Director(Id, Name) VALUES (1, 'Hitchcock');
```

If the same `INSERT` is issued again, it violates the Primary Key constraint and receives error message:

```
Error: UNIQUE constraint failed: Director.Id
```

You can ignore entering the `Id` and the database will create an `AUTOINCREMENT Id`:

```
INSERT INTO Director(Name) VALUES ('Hitchcock');
```

If the same `INSERT` is issued again, it violates the Candidate Key (unique) constraint:

```
Error: UNIQUE constraint failed: Director.Name
```

To check whether data was properly inserted into the table:

```
SELECT *
FROM Director;
```

Now, to first check whether a director is already in the `Director` table or not:

```
SELECT Id
FROM Director
WHERE Name = 'Hitchcock';
```

This will return the `Id` for `'Hitchcock'` if it exists; otherwise it will return nothing. As we already insert this director into `Director` table with `Id=1`, we can insert movies which directed by him into `Movie` table:

```
INSERT INTO Movie(Title,Genre,DirectorId) VALUES ('Psycho','Horror',1);
INSERT INTO Movie(Title,Genre,DirectorId) VALUES ('The Birds','Horror',1);
```

If a movie with a non-existent `DirectorId` (e.g., `DirectorId=2`) is inserted, the Foreign Key constraint is violated:

```
INSERT INTO Movie(Title,Genre,DirectorId) VALUES ('The Birds','Horror',2);

Error: FOREIGN KEY constraint failed
```

Make sure you enter sufficient tuples for each of your tables and also ensure that all your Foreign and Candidate Keys are respected when data are entered in each table.

SELECT. Suppose we want to find all movies that share the same genre as `'Inception'`. We first need to retrieve the genre of `'Inception'` from the `Movie` table and then compare it with the genres of other movies. We can achieve this using table *aliases*, which allow us to refer to the same table in two different roles within a query. We will use `MovieInception` to identify `'Inception'` and retrieve its genre, and `MovieOther` to examine other movies and compare their genres. This leads to the following SQL statement (query):

```
SELECT MovieOther.Title, MovieOther.Genre
```

```
FROM Movie AS MovieInception, Movie AS MovieOther
WHERE MovieInception.Title = 'Inception'
  AND MovieOther.Genre = MovieInception.Genre
  AND MovieOther.Id <> MovieInception.Id;
```

The above example assumes that columns `Genre` and `Title` exists in `Movie` table. Combining tables, and copies of tables using aliases, allows us to build up very complex queries, but they can get quite difficult to understand. Subqueries provide a better way of passing the results of one query into the `WHERE` clause of another. This lets us break the problem down into smaller parts, and then write queries for each part. For example, the above query can be rewritien as:

```
SELECT Title
FROM Movie
WHERE Genre = (
               SELECT Genre
               FROM Movie
               WHERE Title = 'Inception'
              )
  AND Title <> 'Inception'
```

Note in this query the `Genre` in the subquery refers to the genre of `'Inception'`, while the `Genre` outside of the subquery refers to the genre of the results.

Let us now look at a more complex query for finding movies that have the same genre as any movie produced by `'Scorsese'`. The query to find the genre of movies produced by `'Scorsese'` is:

```
SELECT Genre
FROM Movie, Director
WHERE Movie.DirectorId = Director.Id
  AND Director.Name = 'Scorsese'
```

This will typically return more than one result, so we cannot use it as subquery with the equality = operator. So, the following query will *not* work (In SQLite, it works since it selects the first row of the subquery. But it is not what we want in the final results):

```
SELECT Title
FROM Movie
WHERE Genre = (
               SELECT Genre
               FROM Movie, Director
               WHERE Movie.DirectorId = Director.Id
                 AND Director.Name = 'Scorsese'
              )
```

Instead, we can use the `IN` operator for checking whether a value is within a set:

```
SELECT Title
FROM Movie
WHERE Genre IN (
                SELECT Genre
```

```
    FROM Movie, Director
    WHERE Movie.DirectorId = Director.Id
      AND Director.Name = 'Scorsese'
)
```

The most important demonstration is retrieving information about the relationships between two entity sets in relations using `JOIN` statements. For instance, to retrieve the movies made by each director:

```
SELECT Director.Name, Movie.Title
FROM Director
JOIN Movie ON Director.Id = Movie.DirectorId
```

Or to count the number of movies made by each director:

```
SELECT Director.Name, COUNT(Movie.Id) AS NumberOfMovies
FROM Director
JOIN Movie ON Director.Id = Movie.DirectorId
GROUP BY Director.Id, Director.Name;
```

DELETE. It is also possible to delete tuples (rows) from each table. For instance, to delete a specific director:

```
DELETE FROM Director WHERE Name = 'Hitchcock';
```

Tuples (rows) cannot be removed for Foreign Key constraints. If, for example, we were to remove the entry for `'Hitchcock'` from the `Director` table, then the `Id`'s for any movies that have this director would become invalid, and *referential integrity* violated. Because a Foreign Key has been set up between `Movie` and `Director`, SQLite can detect this violation and stop deleting directors who have movies in the `Movie` table:

```
Error: foreign key mismatch
```

UPDATE. It is also possible to update a tuple (row) of a table by `UPDATE` statements. Suppose we had entered `'Hitchcock'`'s name incorrectly and we need to fix it:

```
UPDATE Director SET Name = 'Hitchcock' WHERE Name = 'Hitchkok';
```

It is very important to get the `WHERE` clause on an `UPDATE` statement right. Running an `UPDATE` without a `WHERE` clause will change all of the rows in a table. For example, (*don't do this!*) the command:

```
UPDATE Director SET Name = 'Welles';
```

will change all the directors' name to `'Welles'`. If you want to check the `WHERE` clause, you can try it out with a `SELECT` statement first, to make sure that it returns the rows you expect. For example:

```
SELECT *
FROM Director
WHERE Name = 'Welles';
```

10

4.5. User Interface for Data Entry and Retrieval

Although SQL provides a powerful way to query and modify data, end users should not be expected to interact with the system by writing SQL statements. Such interaction requires technical knowledge and increases the risk of errors, including accidentally deleting tuples, modifying important information, or even removing entire relations. Instead, the system should provide a user-friendly interface that allows users to perform common tasks through forms, buttons, search fields, and other intuitive controls. The interface should validate user input and restrict operations, making the system safer, more accessible, and easier to use. For your team project, you should develop a simple web-based or desktop application that interacts with your database. Rather than entering SQL statements manually for DML operations. For example, an application should provides:

- Data entry: A form for adding a new director and the movies made by that director.
- Data retrieval: Search and filter controls for finding directors by name or finding movies by genres.
- Data update/delete: A form to update the information or delete an exising director or movie.

Important: For this step and in this course, focus on a working demonstration rather than a polished product.

4.6. Implementation (Code)

By implementation, we mean translating your proposed data model into a working database. This includes creating the database and its tables, defining Primary and Foreign Keys and other constraints, inserting sample data, and retrieving and modifying records using SQL statements such as `SELECT`, `INSERT`, `UPDATE`, and `DELETE`. Your implementation should follow the FAIR Guiding Principles[10], which promote making digital project objects Findable, Accessible, Interoperable, and Reusable (by humans and machines/agents). For this course, this means making a reasonable effort to ensure that another person can locate your database implementation, understand its structure, set it up, and reproduce your results. Your implementation should:

- **Be publically accessible online.** Provide SQL scripts and other necessary files in a repository or submission package that can be accessed for evaluation.
- **Follow an organized structure and architecture.** Organize SQL statements and related files into clearly named files or sections. For example, separate database and table creations from data insertion, data retrieval or modification where appropriate. In addition, as shown in Figure 1, include an architectural diagram illustrating the main layers of your system and how they interact. For a typical data modeling application, these layers may include the *user interface layer*, through which users interact with the system; the *application or logic layer*, which processes requests and communicates with the database; and the *data layer*, which manages the store and retrieve data. Clearly indicate the role of each layer and the flow of information between them. Your architecture should be consistent with your proposed solution and help readers understand how the database fits into the system.
- **Be reproducible.** Provide the SQL scripts and instructions needed to create and populate the database from scratch. Another person should be able to follow your instructions to recreate the tables, insert the sample data, and execute your queries without relying on an undocumented, pre-existing database.
- **Be consistent with your proposed data model.** The implemented tables, columns, keys, relationships, and constraints should reflect your proposed data model. Ensure that Keys and other constraints are defined appropriately to preserve data integrity.
- **Demonstrate the required operations.** Include SQL statements that demonstrate the insertion of sample records, retrieval of data using `SELECT` queries, and modification or deletion of records using `UPDATE` and `DELETE`, as appropriate to the team project requirements. Include examples of joins, aggregation, or other queries where required. In addition, as shown in Figure 2, provide screenshots of your web/desktop

Commented [A1]: At your team's discretion. We do not require public repo for this course due to students' personal information and privacy, but it should be accessible to instructors and markers for evaluation purposes.

[10] The FAIR Guiding Principles for scientific data management and stewardship: https://www.nature.com/articles/sdata201618

application demonstrating each main operation through its user interface. The screenshots should show the application in use, the operation being performed, and the resulting output or changes to the data. Where appropriate, include examples of adding a tuple, viewing or searching them, updating or deleting an existing tuple, etc. Ensure that the screenshots are readable, clearly labeled, and consistent with your implemented database and the SQL statements provided.

- **Include clear documentation.** Provide a `README` or equivalent documentation explaining how to set up the database, execute the SQL scripts, and reproduce the demonstrated results. Specify the DBMS used and any required configuration or setup steps.

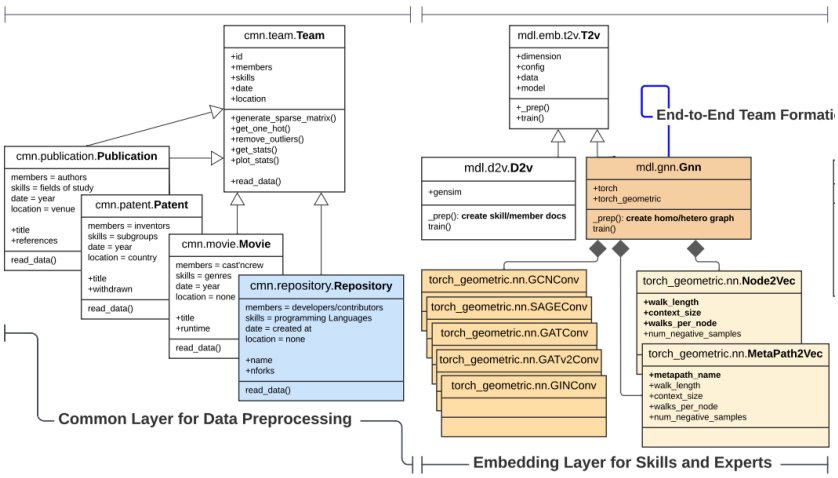


Figure 1. An overview of our system architecture. Coloured components are our system's main contributions.

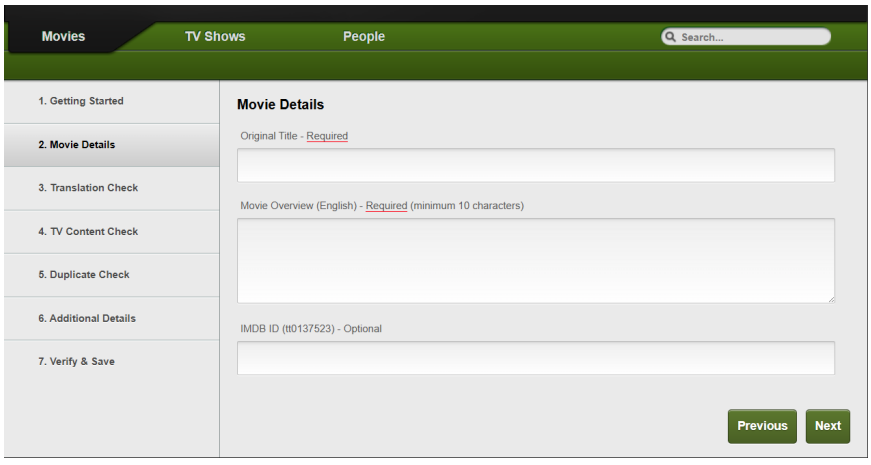


Figure 2. The user interface to add a new movie.

4.7. Submission Guidelines

4.7.1. Deliverables

- Submission must be written in English, in the current ACM two-column conference format in LaTeX. Overleaf[11] templates are available from the ACM Website[12]. Use the *sigconf* proceedings template.
- The submission must be *exactly* 2 pages (text) + 1 page (list of all relations and constraints after ER-to-Relational mapping) in length, no more and no less, including author list, figures, tables, and references, with unlimited pages for C: AI-Assisted Editing appendix, and must be authored by all team members, with the graduate student listed last in the author list. We understand that fitting all the required content into exactly one page can be challenging. However, this is intentional: an important part of communication is learning how to retouch, refine, condense, and prioritize information without losing its essential meaning. We recommend initially writing a longer version (e.g., 5-10 pages) and then gradually reducing and retouching it until you can effectively communicate the key ideas within the required three-page limit.
- The submission must be a single zip file named: 4_Relational_firstname1_firstname2_[...].zip, e.g., 4_Relational_Kovvy_Hossein.zip, containing:
  - The LaTeX source files and any required supplementary files;
  - The compiled three-page PDF file.

  A *sample* is attached to this manual and is also available here[13] (4_Relational.tex).

4.7.2. Team and Individual Contributions

- Each team must maintain a Git-based repository (e.g., GitLab, GitHub, …)[14] for the project and linked inside the report in title's footnote and the abstract. The repository must be associated with the individual accounts of all team members. The repository will be used to document the development of the project and to make individual contributions traceable. The repository should be used throughout the project rather than only immediately before a submission deadline. The instructor and/or marker may consider the repository's commit history, pull requests, issues, documentation, and other project records when assess individual contributions. The number of commits alone will not be used as a measure of contribution. The quality, significance, and nature of each member's contributions will also be considered.
- All team members are expected to make substantive contributions, and no member should be limited solely to coordination or project management.

2.2.1. Marking Scheme

- 10% Author list, style, format, repo, naming convention, …
- 20% Report on the main relations and respective DML operations, mainly the `SELECT` queries
- 20% Illustrative figures of the system architecture and user interfaces for the web/desktop application
- 50% Code (FAIR-compliant) Please also see Appendix A: Recommended Code Structure.
  - 20% Architecture (Layers, driver, …).
  - 10% README (Project bio, installation, licenese, …)
  - 10% Quickstart script to setup the application for demonstration
  - 10% Reproducible/Repeatable demonstration

Commented [A2]: Making the repo public is at your team's discretion. We do not require public repo for this course due to students' personal information and privacy, but it should be accessible to instructors and markers for evaluation purposes.

[11] https://www.overleaf.com/gallery/tagged/acm-official
[12] https://www.acm.org/publications/proceedings-template
[13] https://www.overleaf.com/read/vtggnmsqqfqy
[14] Web-based applications and cloud services may collect, use and/or disclose your personal information and store or access that information outside of Canada. Personal information includes, but not limited to, identifiable individual's information such as your name or your email address. At UVic, GitLab at gitlab.uvic.ca is highly recommended for coursework, research and development projects. For more information, see https://www.uvic.ca/systems/services/pages/gitlab.php. At your team's discretion, you may use GitHub to store information about your project, but we do not require nor recommend storing any personal identifying information. Also, we do not require public repo, but it should be accessible to instructors and markers for evaluation purposes.

Appendix A: Recommended Codebase Structure

This is a template suggested for project implementation based on my experience in data science and software development projects. It is important to note that

- This is a general template and should be adjusted for any projects' need,
- The main goal is to make your codebase useful enough that other researchers can reproduce and replicate your work, fork your repository, build on top of it, and happily choose it as a baseline; without needing your involvement*!* The more people who use, extend, and cite your code, the more visible and impactful your work becomes.
- Your suggestions are more than welcome.

README.md

1. Intro [Overview]
2. Illustrative Sample
3. Prerequisite [Package Requirements]
4. Installation Guide
5. Quickstart
6. Citation
7. License

requirements.txt *#for dependencies of this project and other libs and packages*

\data

- \raw *#the original dataset for the testbed*
- \preprocessed *#the final dataset required for this project*

\src

- \libs *#any opensource library that are customized for this project*
- \cmn *#any common code to other parts of the project*
- \ui *#any code related to the user interfaces*
- \biz *#any code related to business/system decisions to convert input/output for store/retrieve from database*
- \dal *#any code related to access the database for DML operations*
- \dal\models\er *#any artifact related to conceptual modeling using entity-relationship*
- \dal\models\relational *#any artifact related to relational modeling*
- \dal\models\relational\sql\ *#any sql script for DDL and DML*
- \dal\models\semantic *#any artifact related to semantic web modeling including ontologies*
- main.[py, c, …] *#the main entry of the project's pipeline*

\doc

- *any documentation, figure, screenshot, diagram, additional instruction, …*